



OAC 2025/2 – Laboratório 4
- CPU RISC-V Pipeline -

Grupo A8:

Gabriel de Medeiros Matos - 242001491

Gabriel Fonseca Pimenta - 242043428

Giovanni Daldegan - 232002520

Ricardo de Carvalho Nabuco - 231021360

Tauã Valentim de Albuquerque Martins Frade - 231021389

1. Processador Pipeline – ISA RISC-V32I reduzida

1.1. Registradores de transição de estágios

Para dividir o processador nos 5 estágios de pipeline IF, ID, EX, MEM e WB, foram adicionados 4 registradores de transição: IF_ID, ID_EX, EX_MEM e MEM_WB.

Cada registrador armazena dados e/ou sinais de controle utilizados nos estágios seguintes. A imagem abaixo detalha as seções de cada registrador.

```
/*  
* Seções dos registradores de transição  
*  
* IF_ID: 0:31 PC, 32:63 PC4, 64:95 Instr  
* ID_EX: 0:31 PC4, 32:36 rd, 37:68 Dado1, 69:100 Dado2, 101:132 Imm, 133:135 WB, 136:137 MEM, 138:154 EX  
* EX_MEM: 0:31 PC4, 32:36 rd, 37:68 ResULA, 69:100 Dado2, 101:132 Imm, 133:135 WB, 136:137 MEM  
* MEM_WB: 0:31 PC4, 32:36 rd, 37:68 ResULA, 69:100 MemLeitura, 101:132 Imm, 133:135 WB  
*  
* sinais EX: 0:1 OrigAULA 2:3 OrigBULA 4:5 opULA 6 jalr 7:9 funct3 10:16 funct7  
* sinais MEM: 0 LeMem 1 EscreveMem  
* sinais WB: 0 EscreveReg 1:2 OrigReg  
*/
```

Cada “registrador de transição” é um módulo. Tecnicamente, eles recebem as entradas correspondentes aos dados e sinais que armazenam, guardando esses valores em registradores internos de cada módulo.

Alguns dos sinais armazenados e seus respectivos registradores são:

- “Instr”, a instrução lida no estágio IF e decodificada em ID;
- “Imm”, o imediato gerado em ID e propagado até o estágio WB;
- “PC4”, valor de PC + 4 calculado em ID e propagado até WB;
- “ResULA”, resultado da ULA obtido em EX e propagado até WB

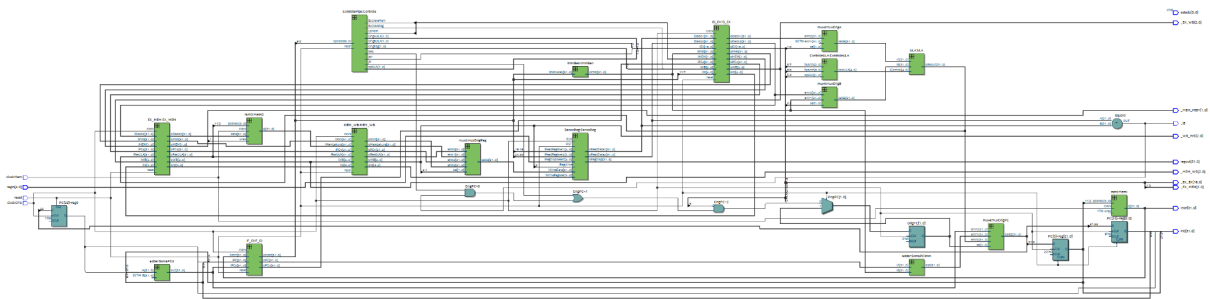
Quanto aos sinais de controle:

- “WB” guarda sinais usados no estágio WB: “EscreveReg” (*enable* de escrita no banco de registradores) e “OrigReg” (origem do dado)
- “MEM” guarda os sinais “LeMem” e “EscreveMem”
- “EX” guarda todos os sinais usados no estágio EX, o que inclui “opULA” (operação passada ao controle da ULA), “jalr” (indicando que o pulo da instrução será realizado no estágio EX), entre outros.

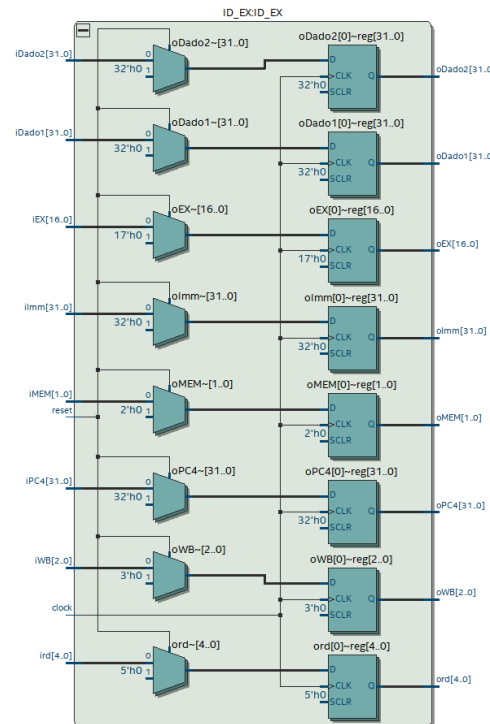
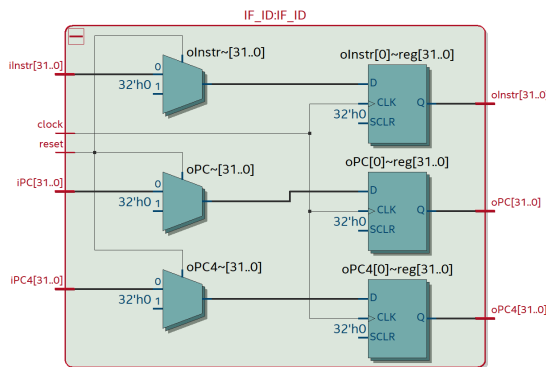
1.2. Processador completo

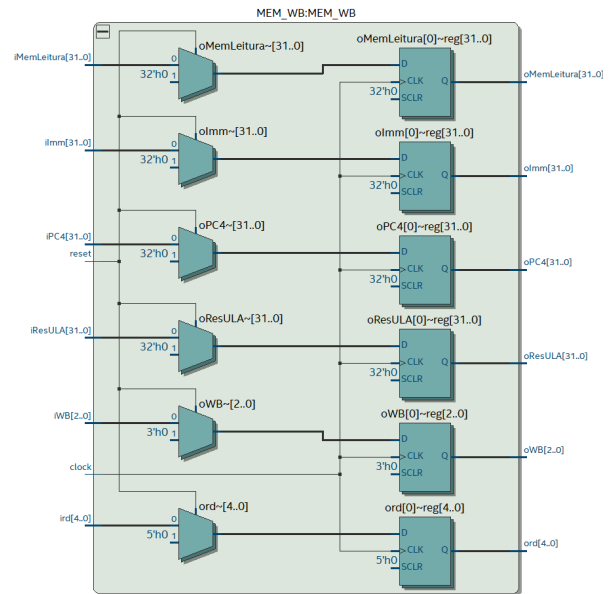
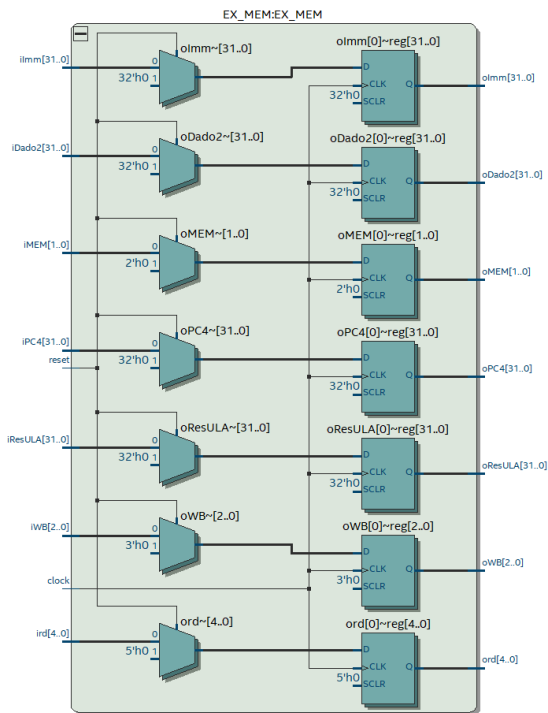
a) Visualizações – RTL view

Pipeline completo

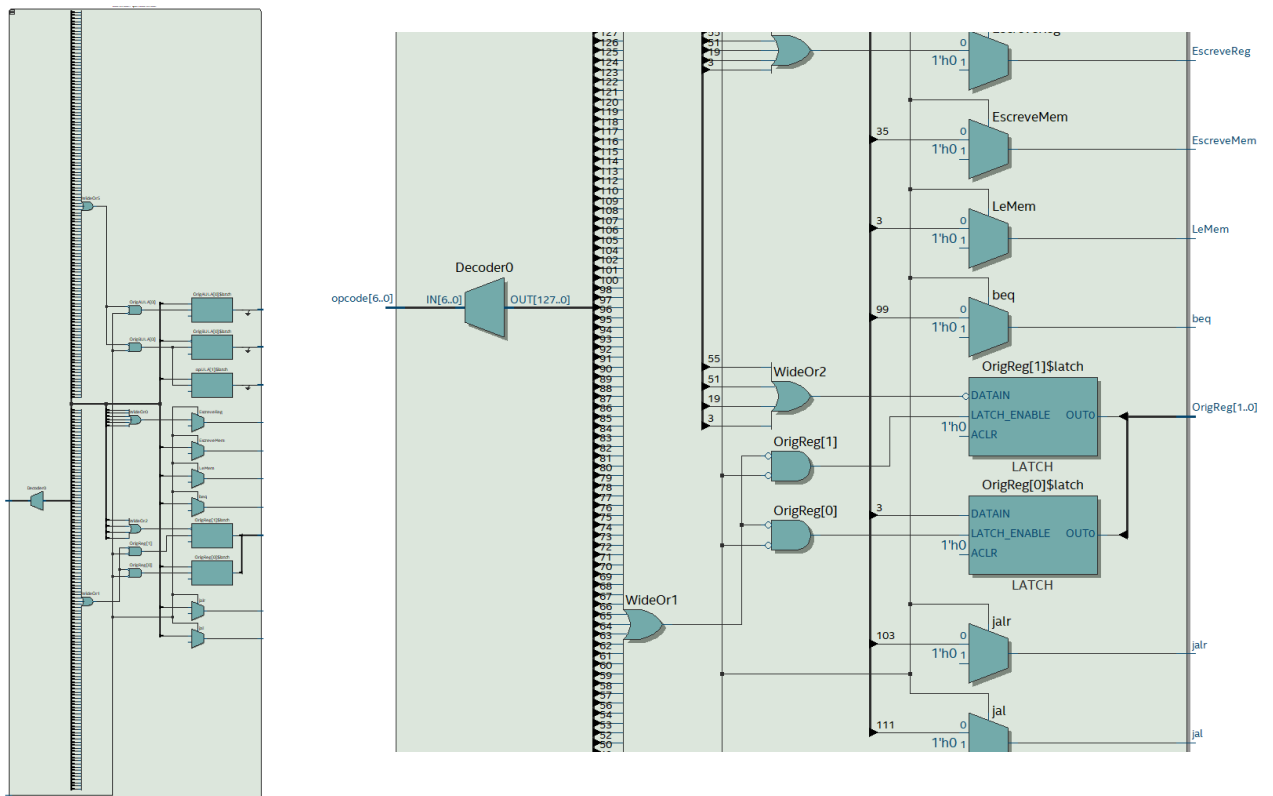


Registadores de transição IF_ID, ID_EX, EX_MEM, MEM_WB (em ordem)

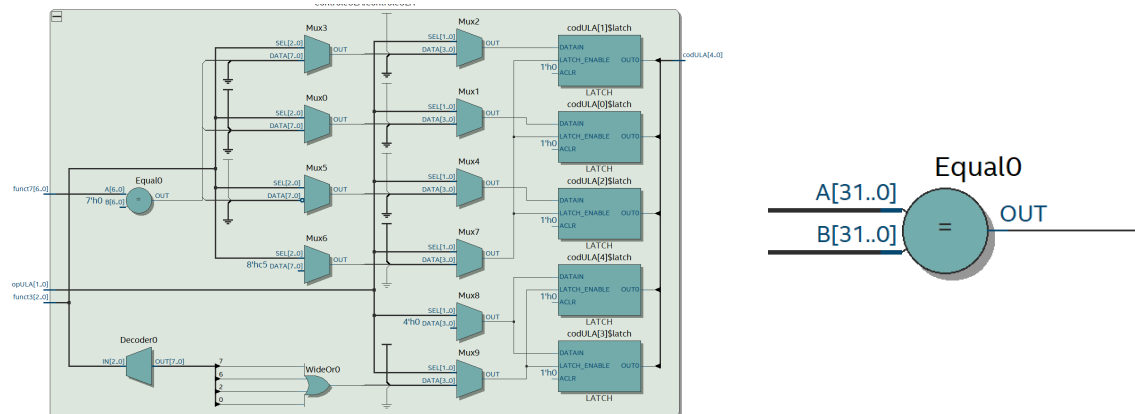




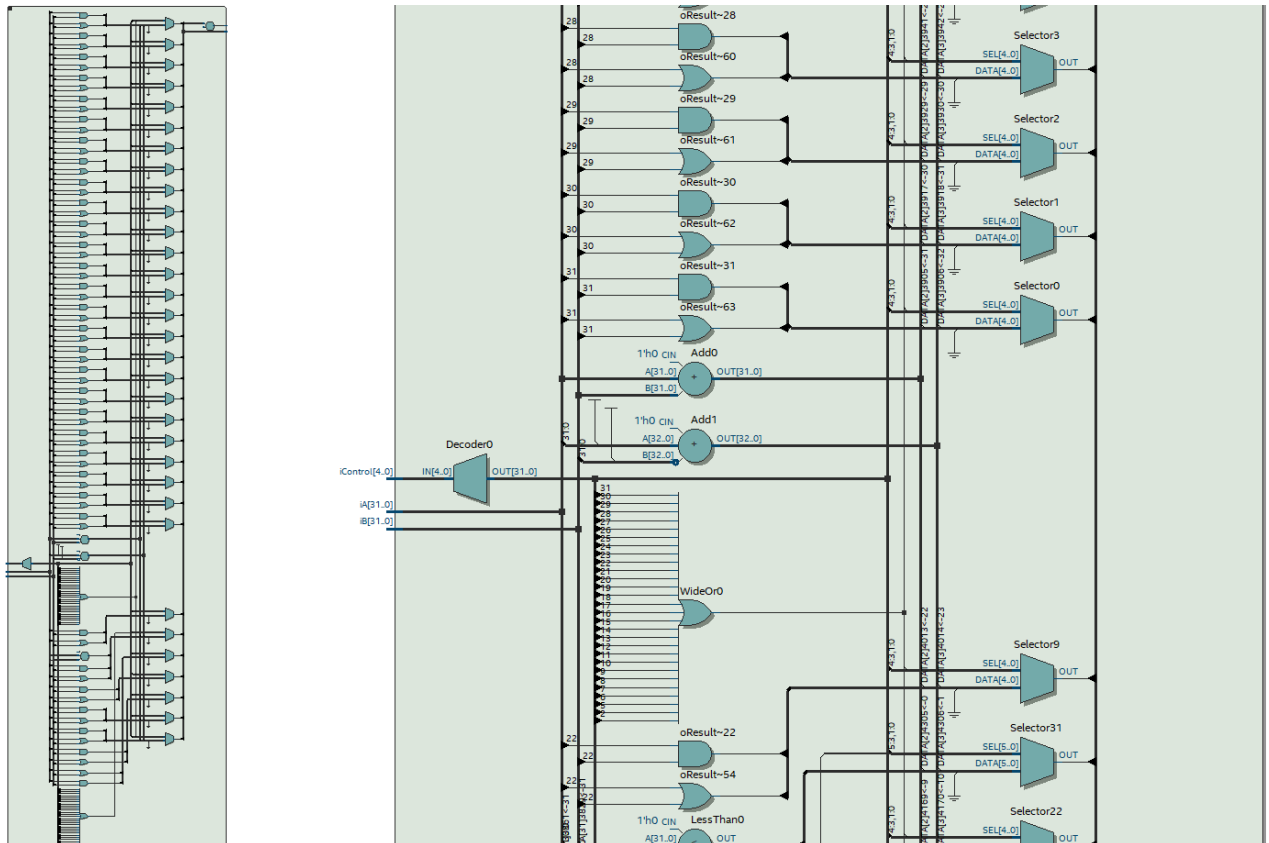
Controle (completo e com zoom, lado a lado)



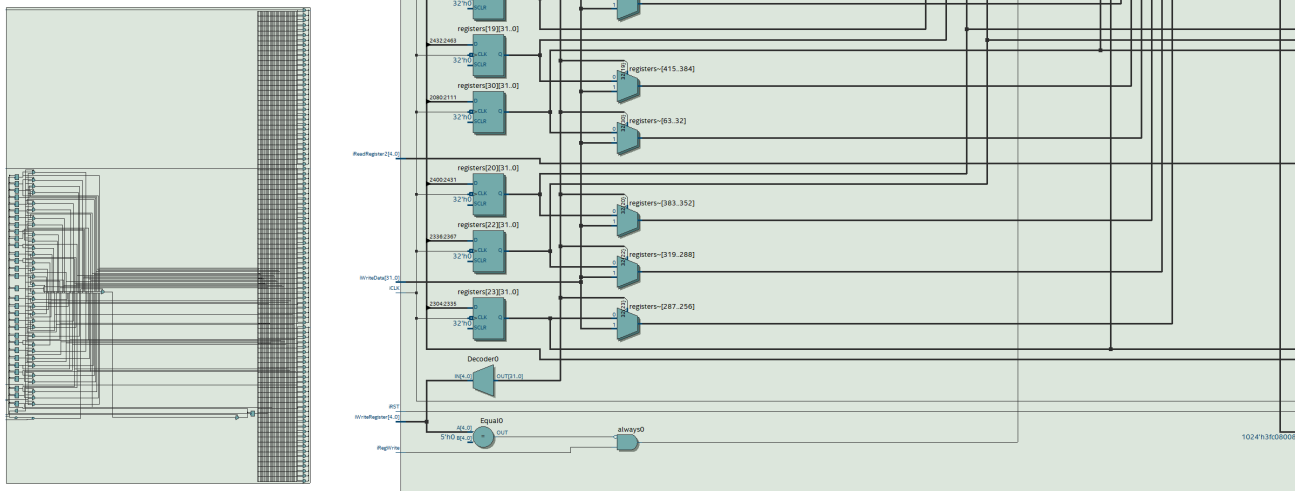
Controle da ULA e comparador de rs1 e rs2



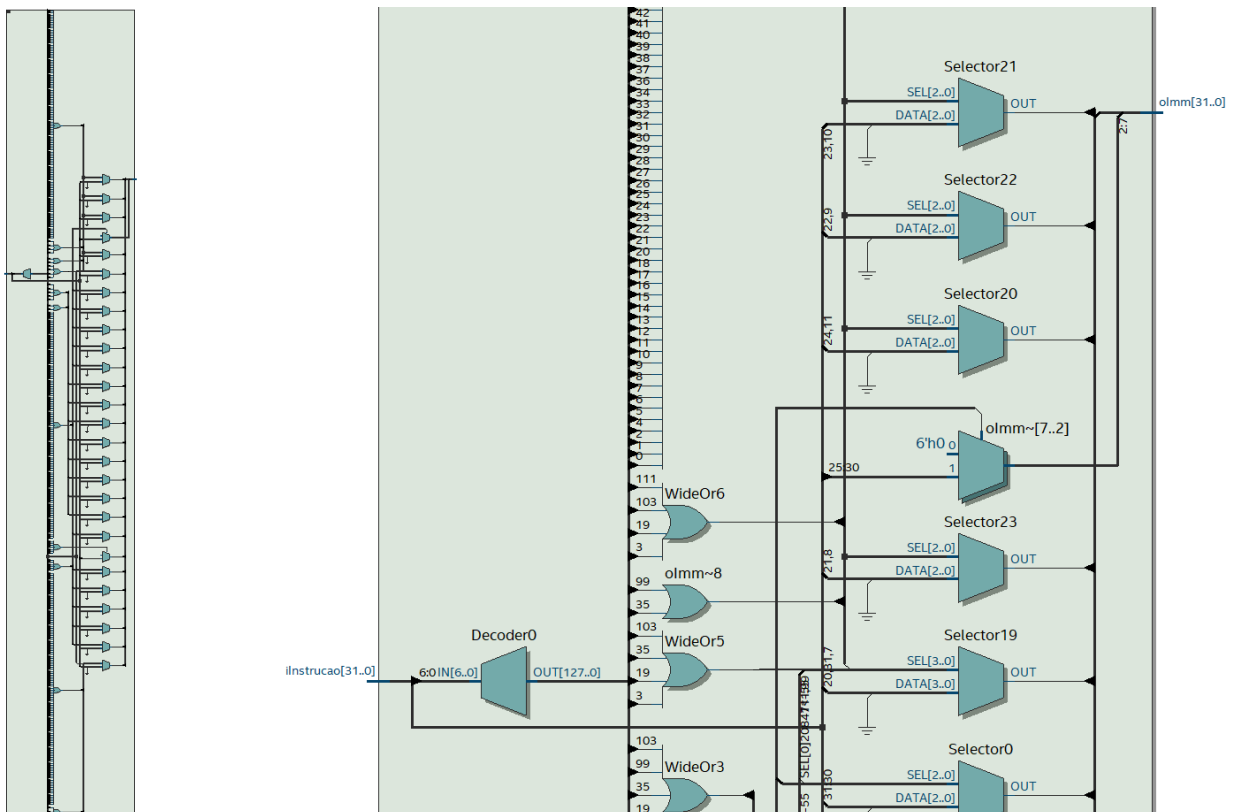
ULA (completa e com zoom)



BancoReg (completo e com zoom)



ImmGen (completo e com zoom)



b) Requisitos físicos e temporais

Requisitos físicos

Flow Summary	
<<Filter>>	
Flow Status	Successful - Wed Dec 10 18:19:00 2025
Quartus Prime Version	24.1std.0 Build 1077 03/04/2025 SC Lite Edition
Revision Name	TopDE
Top-level Entity Name	TopDE
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	1,462 / 32,070 (5 %)
Total registers	1793
Total pins	108 / 457 (24 %)
Total virtual pins	0
Total block memory bits	65,536 / 4,065,280 (2 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Requisitos temporais

- Slack de setup

	Clock	Slack	End Point TNS
1	CLK	0.720	0.000

- Slack de hold

	Clock	Slack	End Point TNS
1	CLK	0.055	0.000

- Frequência máxima

	Fmax	Restricted Fmax	Clock Name	Note
1	53.88 MHz	53.88 MHz	CLK	

- Datasheet

	Data Port	Clock Port	Rise	Fall	Clock Edge	Clock Reference
1	Reset	CLK	3.871	5.285	Rise	CLK
2	Reset	CLK	1.817	2.527	Fall	CLK

c) Simulação e correção de hazards

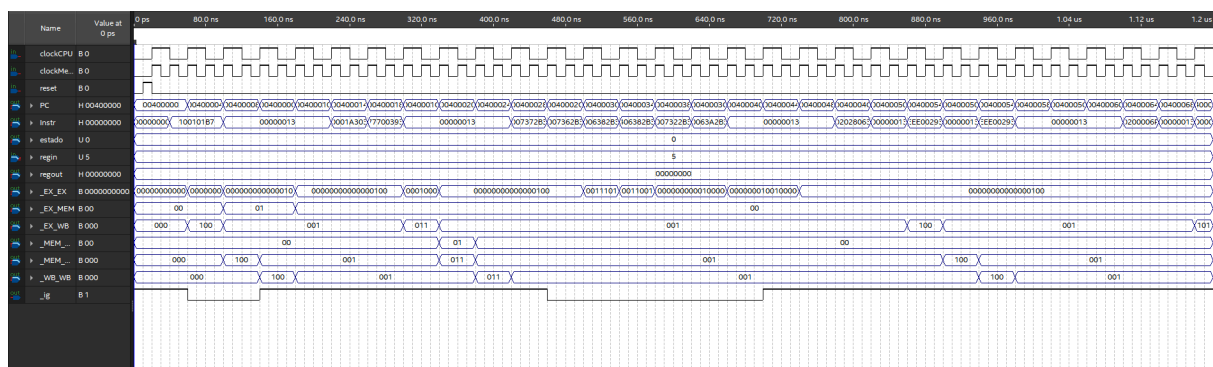
O processador não possui detecção de hazards, forwarding ou outras estratégias de otimização. Para evitar falhas na execução do programa `del.s`, foram adicionadas instruções `nop` no programa (codificada no RARS como `addi zero, zero, 0`).

Toda instrução que escreve no banco de registradores (no estágio WB) e apresenta hazard em relação à próxima instrução precisa ser seguida por 3 instruções `nop`, para que o estágio WB da instrução “pré-hazard” alinhe com o estágio IF da instrução “pós-hazard”; assim, no estágio ID da última instrução, o resultado da última já foi gravado no banco de registradores.

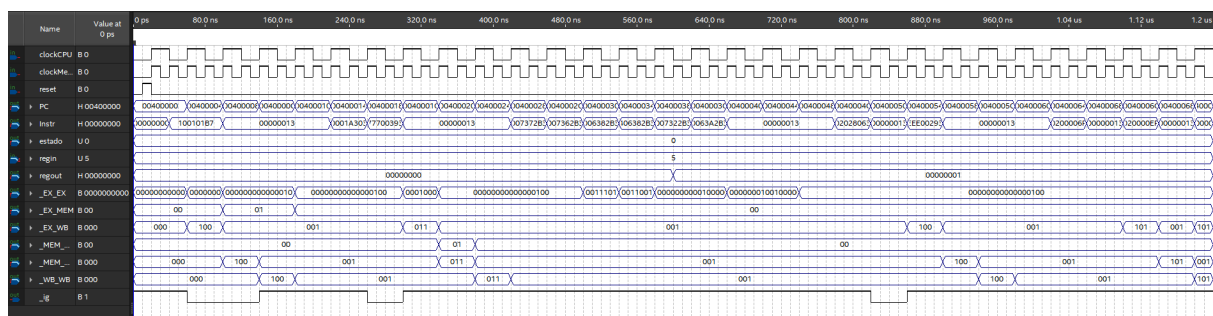
Além disso, considerando a ISA reduzida e a ausência de forwarding, os hazards de controle ocorrem apenas com instruções de pulo. O pulo condicional de um `beq` tomado e o pulo `jal` são realizados no estágio ID, onde a instrução é decodificada e `rs1` e `rs2` são comparados para o *branch*, indicando que basta o flush de uma instrução para corrigir o hazard; então, foi adicionada uma `nop` após cada uma dessas instruções. Já `jalr` precisa que o resultado da ULA (`rs1 + Imm`) seja gravado no banco de registradores; então foram adicionadas duas `nop`, permitindo que o pulo ocorra após o estágio EX sem hazards.

No total, com as `nop` incluídas no programa, a execução de `del.s`, são 42 instruções.

- Simulação funcional



- Simulação temporal



d) Frequência máxima de clock

	Fmax	Restricted Fmax	Clock Name	Note
1	53.88 MHz	53.88 MHz	CLK	

A frequência máxima observada no processador foi 53.88 MHz

1.3. Comparação dos processadores – ISA RV32I reduzida

A tabela abaixo compara os requisitos físicos dos três processadores desenvolvidos durante o semestre.

Processador	Uniciclo	Multiciclo	Pipeline
Logic utilization	1.136	1.508	1.462
Total registers	1.296	1.406	1.793
Total block memory bits	65.536	65.536	65.536

- O Uniciclo é o mais simples, com a menor quantidade de lógica e registradores, pois toda a instrução é executada em um único ciclo sem subdivisões.
- O Multiciclo apresenta maior utilização de lógica porque possui uma máquina de estados responsável por coordenar cada etapa da instrução, o que aumenta a complexidade do controle.
- O Pipeline possui o maior número de registradores porque cada estágio precisa armazenar resultados intermediários.

A tabela abaixo compara os requisitos temporais dos três processadores desenvolvidos durante o semestre.

Processador	Uniciclo	Multiciclo	Pipeline
Slack de Setup	0.136 ns	1.218 ns	0.720 ns

Slack de Hold	1.042 ns	0.549 ns	0.055 ns
---------------	----------	----------	----------

Comentário sobre os resultados temporais:

- O Uniciclo sofre no tempo porque toda a lógica é resolvida dentro de um único ciclo, gerando um caminho crítico grande.
- O Multiciclo divide a execução em etapas menores, reduzindo o caminho crítico e melhorando bastante o slack.
- O Pipeline, mesmo dividindo a execução em estágios, apresentou o menor slack de setup entre os três.

Isso indica que um dos estágios ficou pesado demais, desbalanceando o fluxo e limitando a frequência máxima do processador.

Comentário sobre hazards e forwarding:

O pipeline do laboratório funciona sem detecção de hazards e sem técnicas de forwarding.

Isso significa que:

- Ele não identifica automaticamente conflitos entre instruções dependentes.
- Ele não envia resultados adiantados para estágios seguintes.
- Ele depende totalmente de bolhas (NOPs) inseridas manualmente para evitar erros.

Se o pipeline tivesse detecção automática de dependências e forwarding, ele precisaria de menos bolhas, aproveitaria melhor os estágios, aumentaria o fluxo de instruções e equilibraria o caminho crítico. Isso permitiria que o processador operasse em frequências bem mais altas.

1.4. Melhor tempo de execução

	Uniciclo	Multiciclo	Pipeline
Nº instruções	19	19	42
CPI	1	3,73	1,135
Período de clock	20 ns	5 ns	10 ns
Tempo de Execução	380 ns	354,35 ns	476,756 ns

Concluimos então que, devido ao pipeline do laboratório funcionar sem detecção de hazards e sem técnicas de forwarding, ele não pode atingir o seu potencial total, fazendo com que o processador Multiciclo seja o mais rápido na execução do programa de l.s.

2. Vídeo de apresentação

[A8 - Lab 4-20251210_232727-Gravação de Reunião.mp4](#)

3. Referências bibliográficas

dave_59. **Elegant way to define range of bits: Post #8**. [S. l.], 15 dez. 2021. Disponível em: <https://verificationacademy.com/forums/t/elegant-way-to-define-range-of-bits/39884/8>.

Acesso em: 6 dez. 2025.

vlsifacts. **Understanding Pipeline Design in Verilog**: How to Stage Data Across Clock Cycles for High Performance. [S. l.], 5 jul. 2025. Disponível em: <https://vlsifacts.com/understanding-pipeline-design-in-verilog-how-to-stage-data-across-clock-cycles-for-high-performance/>. Acesso em: 8 dez. 2025.